

Rising Waters: A Machine Learning Approach to Flood Prediction

Project Description:

Rising Waters is a machine learning-powered flood prediction system built to give disaster management teams and meteorologists earlier warning of localized flooding. The platform takes five weather parameters — cloud cover, annual rainfall, and seasonal rainfall totals for Jan–Feb, Mar–May, and Jun–Sep — and evaluates them against patterns learned from historical flood data to classify the input as either a flood risk or a safe condition.

The web application is built with Flask and a lightweight HTML/CSS/JavaScript frontend styled as a dark-themed dashboard. Users land on a Home page describing the project, move to a Prediction Input page to submit weather readings, and are routed to one of two outcome pages — a flood-risk warning screen or an all-clear screen — based on the model's output. The trained model and its feature scaler are serialized with Joblib (`floods.save` and `transform.save`) and loaded once at application startup for fast, consistent predictions.

Model development was carried out in a Jupyter notebook (`main.ipynb`) using a historical weather/flood dataset (115 records, `flood_dataset.xlsx`), comparing Decision Tree, Random Forest, K-Nearest Neighbours, and Gradient Boosting approaches before saving a selected model for deployment.

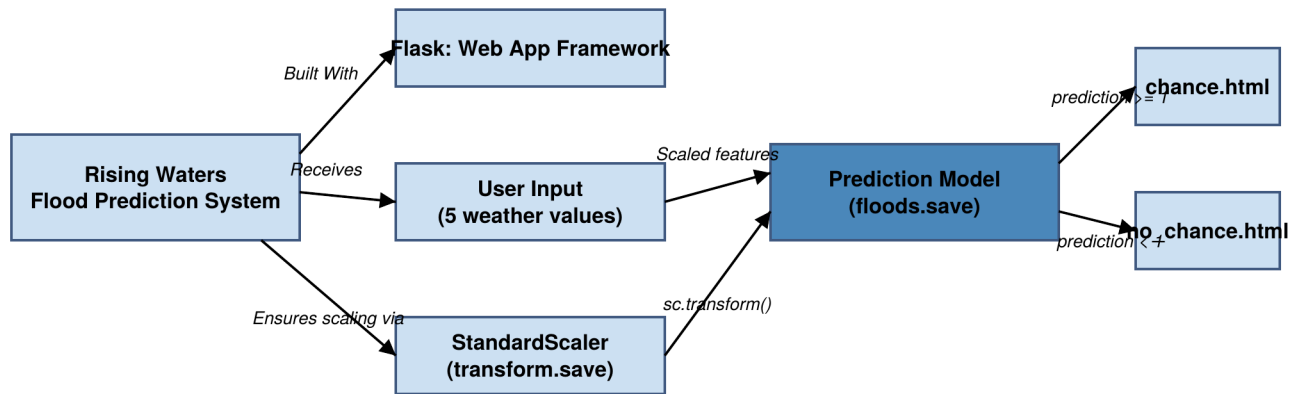
Scenarios

Scenario 1: A meteorologist monitoring a flood-prone district enters the current cloud cover percentage and seasonal rainfall totals into the Prediction Input page. The system evaluates the readings and routes the user to the Critical Flood Risk page, prompting early evacuation planning.

Scenario 2: A disaster relief coordinator checks several regions in turn during monsoon season, submitting each region's rainfall and cloud cover readings. Regions with stable rainfall patterns are routed to the Environmental Status Secure page, letting the coordinator focus resources on higher-risk areas first.

Scenario 3: A government analyst re-runs historical weather records through the Predict Floods tool to sanity-check how the deployed model classifies known past events, comparing the app's output against the recorded outcome for each case.

Technical Architecture:



Description: Rising Waters follows a simple two-tier architecture. The frontend is a set of server-rendered HTML pages (Home, Prediction Input, and two result pages) styled with a shared dark theme in `main.css`, with `main.js` handling a basic loading state on form submit. The Flask backend exposes three routes (`/`, `/predict`, `/submit`) and, at startup, loads a pre-trained classification model and a `StandardScaler` from disk using `Joblib`. On submission, the five form values are scaled using the saved scaler and passed to the model; the resulting prediction determines whether the user is shown the flood-risk warning page or the all-clear page.

Note: This project does not use a database — the historical dataset (`flood_dataset.xlsx`) is only used offline, during model training in the notebook. No ER Diagram is required since no live data store is part of the deployed application.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Scikit-learn / Model Persistence Familiarity:** [Scikit-learn Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Pandas & NumPy for Data Handling:** [Pandas Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)

Project Workflow:

Milestone 1: Model Selection and Data Preparation

- Activity 1.1: Load and explore the historical flood dataset (flood_dataset.xlsx).
- Activity 1.2: Select feature columns and the target column; split into train/test sets.
- Activity 1.3: Scale features using StandardScaler.
- Activity 1.4: Train and compare Decision Tree, Random Forest, KNN, and Gradient Boosting models.

Milestone 2: Flask Backend Development

- Activity 2.1: Load the saved model and scaler with Joblib at application startup.
- Activity 2.2: Define routes for the Home page, Prediction Input page, and the /submit prediction endpoint.

Milestone 3: Frontend Development

- Activity 3.1: Build the Home, Prediction Input, and two result page templates.
- Activity 3.2: Style the interface with a shared dark theme (main.css) and add form-submit feedback (main.js).

Milestone 4: Local Testing and Deployment

- Activity 4.1: Run and test the application locally with the Flask development server.
- Activity 4.2: Prepare the application for scalable deployment (e.g., on IBM Cloud).

Milestone 5: Model Validation and Known Issues

- Activity 5.1: Review notebook results against the deployed model's behavior.
- Activity 5.2: Document known issues found during review and recommended fixes.

Milestone 6: Conclusion

Milestone 1: Model Selection and Data Preparation

This milestone covers loading the historical flood dataset, selecting features and target, preprocessing, and training multiple candidate models for comparison.

Activity 1.1: Load and Explore the Dataset

The dataset (`flood_dataset.xlsx`) contains 115 records with 11 columns: Temp, Humidity, Cloud Cover, ANNUAL, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, avgjune, sub, flood. There are no missing values. A correlation heatmap and distribution plots were used to understand feature relationships before modeling.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_excel('flood_dataset.xlsx')
df.describe()
df.isnull().any()

fig = plt.gcf()
fig.set_size_inches(15, 15)
sns.heatmap(df.corr(), annot=True, cmap='summer',
            linewidths=1, linecolor='k', square=True,
            vmin=-1, vmax=1, cbar=True)
```

Activity 1.2: Select Features and Target, Split Data

Five weather columns (Cloud Cover, ANNUAL, Jan-Feb, Mar-May, Jun-Sep) are selected as input features, matching the five fields on the Prediction Input page.

```
x = df.iloc[:, 2:7].values # Cloud Cover, ANNUAL, Jan-Feb, Mar-May, Jun-Sep
y = df.iloc[:, 9].values   # target column

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=10)
```

Note on column selection: `df.iloc[:, 9]` is the **sub** column (index 9), not the **flood** column (index 10). This is flagged and explained in the "Model Validation and Known Issues" section later in this document, along with a recommended fix.

Activity 1.3: Feature Scaling

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.fit_transform(x_test) # see Known Issues: re-fits scaler on test data
```

Activity 1.4: Train and Compare Candidate Models

Four modelling approaches were compared: a Decision Tree Regressor, a Random Forest Regressor, a K-Nearest Neighbours Regressor, and an XGBoost Regressor.

```

from sklearn import tree, ensemble, neighbors
import xgboost as xgb

dtree = tree.DecisionTreeRegressor()
Rf = ensemble.RandomForestRegressor()
knn = neighbors.KNeighborsRegressor()
xgb_model = xgb.XGBRegressor()

y_train_flat = y_train.ravel()
dtree.fit(x_train, y_train_flat)
Rf.fit(x_train, y_train_flat)
knn.fit(x_train, y_train_flat)
xgb_model.fit(x_train, y_train_flat)

```

A helper function named `xgboost()` was also defined to train and evaluate a classifier version of the model, and is called at the end of the cell:

```

from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

def xgboost(X_train, X_test, y_train, y_test):
    model = GradientBoostingClassifier()
    y_train_clean = y_train.ravel().astype(int)
    y_test_clean = y_test.ravel().astype(int)
    model.fit(X_train, y_train_clean)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test_clean, y_pred.astype(int))
    print(f"Accuracy: {accuracy:.4f}")
    return model, y_pred

# Example Usage
xgb_model, xgb_pred = xgboost(x_train, x_test, y_train, y_test)

```

Activity 1.5: Save the Selected Model and Scaler

```

import joblib
joblib.dump(xgb_model, 'floods.save')
joblib.dump(sc, 'transform.save')

```

Milestone 2: Flask Backend Development

The Flask backend (app.py) loads the saved model and scaler once at startup, then exposes three routes: the Home page, the Prediction Input page, and the /submit endpoint that runs the prediction and returns the appropriate result page.

Activity 2.1 & 2.2: app.py — Routes and Prediction Logic

```
from flask import Flask, render_template, request
from joblib import load
import numpy as np

app = Flask(__name__)

# Load model assets
model = load('floods.save')
sc = load('transform.save')

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/predict')
def index():
    return render_template("index.html")

@app.route('/submit', methods=['POST'])
def predict():
    if request.method == 'POST':
        # 1. Capture the 5 weather parameters from the form inputs
        features = [
            float(request.form['v1']),
            float(request.form['v2']),
            float(request.form['v3']),
            float(request.form['v4']),
            float(request.form['v5'])
        ]

        # 2. Structure as a 2D array and apply the saved StandardScaler (sc)
        final_features = [np.array(features)]
        scaled_features = sc.transform(final_features)

        # 3. Pass features to the saved model
        prediction = model.predict(scaled_features)

        # 4. Redirect the user to the appropriate result page based on output
        if prediction >= 1:
            return render_template('chance.html')
        else:
            return render_template('no_chance.html')

if __name__ == '__main__':
    app.run(debug=True)
```

The five form fields (v1–v5) correspond to Cloud Cover, Annual Rainfall, Jan–Feb Rainfall, Mar–May Rainfall, and Jun–Sep Rainfall — the same five columns used to train the model in Milestone 1.

Milestone 3: Frontend Development

The interface consists of four Jinja2 templates sharing a single dark-themed stylesheet, plus a small JavaScript file that gives the submit button a loading state.

Activity 3.1: Home Page (*home.html*)

The landing page introduces the project and links through to the Prediction Input page via a "Launch Predictive Tool" button.

```
<div class="header">
  <div class="logo">FLOODS PREDICTION</div>
  <div class="nav-links">
    <a href="/">Home</a>
    <a href="/predict">Predict Floods</a>
  </div>
</div>
<div class="banner"></div>
<div class="content-wrapper">
  <div class="card">
    <h2>Introduction</h2>
    <p>Flood forecasting is the use of forecasted precipitation and streamflow data ...</p>
    <div style="margin-top: 2rem; text-align: right;">
      <a href="/predict" class="btn">Launch Predictive Tool</a>
    </div>
  </div>
</div>
</div>
```

Activity 3.2: Prediction Input Page (*index.html*)

This page renders a form with five numeric inputs, posting to /submit.

```
<form action="/submit" method="POST">
  <div class="form-grid">
    <div class="form-group">
      <label>Cloud Cover (%)</label>
      <input type="number" step="any" name="v1" required>
    </div>
    <div class="form-group">
      <label>Annual Rain Fall (mm)</label>
      <input type="number" step="any" name="v2" required>
    </div>
    <div class="form-group">
      <label>Jan-Feb Rainfall (mm)</label>
      <input type="number" step="any" name="v3" required>
    </div>
    <div class="form-group">
      <label>March-May Rainfall (mm)</label>
      <input type="number" step="any" name="v4" required>
    </div>
    <div class="form-group">
      <label>June-September Rainfall (mm)</label>
      <input type="number" step="any" name="v5" required>
    </div>
  </div>
  <button type="submit" class="btn">Execute Prediction Model</button>
</form>
```

Activity 3.3: Result Pages (*chance.html* / *no_chance.html*)

Two result templates share the same layout with different styling classes (`.result-display.danger` vs. `.result-display.safe`) to visually distinguish a flood warning from an all-clear message, both linking back to `/predict` to run a new evaluation.

Activity 3.4: Styling and Interactivity (*main.css* / *main.js*)

`main.css` defines a dark theme (navy background, green accent, red danger / green safe result cards) shared by all four pages. `main.js` gives the submit button a loading state while the prediction request is processed:

```
document.addEventListener("DOMContentLoaded", function() {
  const form = document.querySelector("form");
  if (form) {
    form.addEventListener("submit", function() {
      const submitBtn = document.querySelector(".btn");
      if (submitBtn) {
        submitBtn.innerText = "Processing Analytics Engine...";
        submitBtn.style.opacity = "0.7";
        submitBtn.style.pointerEvents = "none";
      }
    });
  }
});
```


Milestone 4: Local Testing and Deployment

Activity 4.1: Local Testing

Run the Flask development server and exercise all three routes end to end:

```
(myenv) D:\projects> python app.py
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000

# then in a browser:
# http://127.0.0.1:5000/          -> Home page
# http://127.0.0.1:5000/predict -> Prediction Input page
# submit the form -> /submit -> chance.html or no_chance.html
```

Activity 4.2: Scalable Deployment

For production use, the project is designed to be deployed on IBM Cloud so that meteorologists and disaster management teams can reach the tool from any location during active monsoon monitoring, rather than relying on a local-only Flask server.

Milestone 5: Model Validation and Known Issues

While preparing this documentation, the trained model and scaler files were inspected directly and compared against the notebook and app.py logic. This section records what was found so the team can address it before final submission or demo.

Issue 1: Target Column Mismatch

`y = df.iloc[:, 9].values` selects column index 9, which is **sub** (a continuous rainfall-difference value), not column index 10, **flood** (the intended 0/1 target). As a result, the saved model was trained to predict **sub** values, not flood risk.

Issue 2: Scaler Re-fit on Test Data

`x_test = sc.fit_transform(x_test)` re-fits the StandardScaler on the test set, overwriting the `mean_ / scale_` statistics learned from the training set. The scaler ultimately saved to `transform.save` reflects test-set statistics rather than training-set statistics.

Issue 3: Model Type Does Not Match Its Label

The function named `xgboost()` actually builds a scikit-learn `GradientBoostingClassifier`, not an XGBoost model. Calling it at the end of the cell (`xgb_model, xgb_pred = xgboost(...)`) reassigns the `xgb_model` variable, overwriting the earlier `XGBRegressor`. This is why `floods.save` loads as a `GradientBoostingClassifier` rather than an XGBoost model.

Issue 4: Reported Accuracy

Because the classifier was trained on **sub** treated as roughly 100 separate integer classes, its notebook-reported accuracy on the held-out test set is **0.0345 (3.45%)**, with precision and recall both around 0.019–0.034. The three regression models (Decision Tree, Random Forest, KNN) scored 0.0, 0.0, and 0.0345 respectively when evaluated with the same classification accuracy metric — a metric that is not well suited to their continuous outputs. These figures reflect the current state of the notebook, not the flood-detection accuracy the project is aiming for.

Practical Effect on the Deployed App

Since **sub** values in the dataset range from about 34 to 983, the condition `if prediction >= 1` in `app.py` is true for effectively every prediction. In its current state, the deployed app would route every submission to the flood-risk warning page (`chance.html`), regardless of the weather values entered.

Recommended Fixes

- Change the target selection to the flood column: `y = df.iloc[:, 10].values`.
- Fix the scaler so the test set is transformed with the already-fitted scaler: `x_test = sc.transform(x_test)` (not `fit_transform`).
- Rename or replace the `xgboost()` helper so it either genuinely trains `xgb.XGBClassifier()`, or rename the function/variable to accurately reflect that it trains a `GradientBoostingClassifier`.
- Re-run model comparison on the corrected binary target and re-save `floods.save / transform.save` from the corrected pipeline.
- Re-test the deployed app end-to-end with a range of known flood and non-flood historical records to confirm both outcome pages can actually be reached.

Milestone 6: Conclusion

Rising Waters delivers a working end-to-end pipeline for flood risk prediction: a Flask web application with a Home page, a Prediction Input page, and two outcome pages, backed by a model and scaler trained on historical weather data and served through a lightweight, dark-themed interface. The application architecture, routing, and frontend are complete and functional.

During documentation review, a target-column selection issue was identified in the training notebook that currently causes the deployed model to predict a different quantity than intended, which in turn causes the application to always route to the flood-risk warning page regardless of input. This is a well-understood, fixable issue: correcting the target column, the scaler fit, and re-saving the model artifacts will align the deployed behavior with the project's intended flood/no-flood classification.

Looking ahead, once the corrected model is retrained and validated, Rising Waters is well positioned for further enhancements: expanding the feature set (e.g., incorporating Temp, Humidity, and Oct-Dec rainfall), collecting more historical records to address class imbalance (99 non-flood vs. 16 flood records in the current dataset), adding a confidence score to each prediction, and deploying the corrected application on IBM Cloud for wider access by disaster management teams.